

Proyecto 1: Pairing Heap

Joaquín Salinas Salas Isaac Simeón Sarmiento Adrian Auqui Perez
Ciencia de la Computación, UTEC, Lima, Perú

Resumen: Este informe explica cómo diseñamos y construimos un "organizador de tareas" que soporta tres operaciones sobre un conjunto de elementos con id y clave: agregar un elemento, sacar el elemento con la clave más chica, y bajar la clave de un elemento ya guardado a partir de su id. Para resolverlo construimos un *pairing heap*: una estructura en forma de árbol donde cada nodo puede tener varios hijos, y que funciona uniendo (fusionando) árboles entre sí. En este informe mostramos cómo está armada la estructura, explicamos por qué cada operación funciona correctamente, y calculamos qué tan rápida es cada una.

1 Introducción

El problema pide simular un conjunto de elementos, cada uno con un id único y un número entero como clave. Sobre este conjunto hay que poder: agregar elementos, sacar el elemento con la clave más chica (con empate por menor id), y bajar la clave de cualquier elemento a partir de su id. La regla principal del proyecto es que no se puede usar un heap binario, ni su versión en arreglo, ni estructuras como `priority_queue` ya armadas en el lenguaje. Esto descarta la solución más común: un heap binario con un arreglo extra de posiciones, que permite encontrar cualquier elemento en $O(1)$ para luego "subirlo" tras bajar su clave.

Como alternativa, elegimos el *pairing heap*, propuesto por Fredman, Sedgewick, Sleator y Tarjan (1986). En vez de ser binario, cada nodo puede tener varios hijos, representados con punteros al primer hijo y al hermano de al lado. Para bajar una clave, en lugar de mover elementos como en un heap binario, se corta una parte del árbol y se vuelve a unir en otro lado.

2 Diseño de la implementación

Representación. Cada nodo (`Node`) guarda su `id`, su `key`, y tres punteros: `child` (primer hijo), `sibling` (siguiente hermano) y `prev` (hermano anterior, o el padre si el nodo es el primer hijo de su lista). Este último puntero permite sacar cualquier nodo de su lista de hermanos en $O(1)$. Toda la estructura se maneja con un puntero `root` (siempre el elemento de menor prioridad) y un arreglo auxiliar `nodeOf[id]` que da acceso directo en $O(1)$ al nodo de cualquier id. La prioridad se decide con `hasHigherPriority`: gana la menor `key`, y en empate la menor `id`.

Merge (`mergeTrees`). Es la operación base de todo lo demás. Se comparan las dos raíces y la que pierde pasa a ser el primer hijo de la que gana: se enlaza como `sibling` del hijo anterior del ganador, y como `child` nuevo del ganador. Solo se reacomodan punteros, en tiempo $O(1)$.

Insertar. Se crea un nodo suelto, se guarda en `nodeOf[id]` y se junta con la raíz actual mediante `mergeTrees`. Como el merge es $O(1)$, insertar también lo es.

Sacar el mínimo (`extractMin`). La raíz siempre tiene la menor prioridad, así que su `id` se guarda y se devuelve. Luego hay que unir todos sus hijos (que quedaron sueltos) en un solo árbol para tener la nueva raíz, usando `mergeChildrenByPairs`: primero se juntan los hijos de a pares, de izquierda a derecha; después esos resultados se juntan de derecha a izquierda hasta quedar en uno solo. Este *pairing* de dos pasadas es lo que hace rápida a la estructura en promedio.

Bajar una clave (`decreaseKey`). Se llega al nodo en $O(1)$ vía `nodeOf[id]` y se cambia su clave. Si es la raíz, no hace falta nada más. Si no, se desconecta de su lista de hermanos con `detachFromParent` (usando `prev` para arreglar los enlaces vecinos) y se junta directamente con la raíz mediante `mergeTrees`. No hace falta subir el nodo nivel por nivel como en un heap binario.

Merge(a, b)

```
1: if a = null then return b
2: if b = null then return a
3: (winner, loser) ← orden según hasHigherPriority(a, b)
4: loser.sibling ← winner.child
5: if winner.child ≠ null then
6:   winner.child.prev ← loser
7: loser.prev ← winner
8: winner.child ← loser
9: winner.sibling, winner.prev ← null
10: return winner
```

Insert(id, key)

```
1: node ← new Node(id, key)
2: nodeOf[id] ← node
3: root ← Merge(root, node)
```

ExtractMin()

```
1: minId ← root.id
2: root ← MergeChildrenByPairs(root.child)
3: liberar nodo antiguo de root
4: return minId
```

DecreaseKey(id, v)

```
1: node ← nodeOf[id]
2: node.key ← v
3: if node ≠ root then
4:   DetachFromParent(node)
5:   root ← Merge(root, node)
```

3 Por qué el algoritmo funciona bien

Se mantienen dos reglas en cada operación. **Regla del heap:** la clave de cada nodo es menor o igual (con desempate por id) que la de sus hijos, por lo que la raíz siempre es el elemento de menor prioridad. **Regla de estructura:** cualquier nodo distinto de la raíz se alcanza desde `root` siguiendo solo punteros `child` y `sibling`, y su `prev` siempre apunta a quien debe actualizarse si se desconecta.

Merge respeta ambas reglas porque solo compara un par de raíces y cuelga al perdedor como hijo del ganador, sin tocar el resto del árbol; como ambos cumplían la regla del heap antes de la fusión, el resultado también la cumple. *Insert* es un caso particular de *merge* con un nodo sin hijos, así que hereda su correctitud. *ExtractMin* funciona porque, tras sacar la raíz, cada hijo queda siendo un heap válido por su cuenta, y el *pairing* de dos pasadas los une usando solo *merge*; por inducción, el árbol final también cumple la regla. *DecreaseKey* funciona porque el único problema que puede aparecer al bajar una clave es con el antiguo padre del nodo (nunca con sus hijos, ya que la clave solo baja); al desconectarlo se elimina ese problema, y al fusionarlo con la raíz el árbol vuelve a ser válido. El desempate por id se aplica siempre igual dentro de *merge*.

4 Qué tan rápido es

Se usa un análisis amortizado con función potencial (se puede ver la demostración completa en Fredman et al. [1]), que define un "potencial" del heap según su tamaño y forma.

Operación	Tiempo promedio
Insert	$O(1)$
Merge	$O(1)$
ExtractMin	$O(\log n)$
DecreaseKey	$O(\log n)$

Insert y Merge son $O(1)$ siempre, no solo en promedio: cada una hace una cantidad fija de pasos, sin importar el tamaño del heap.

ExtractMin es $O(\log n)$ en promedio. El costo real depende de cuántos hijos tenía la raíz sacada (hasta $O(n)$ en el peor caso), pero el *pairing* de dos pasadas baja el potencial lo suficiente para que, sobre m operaciones seguidas, el costo total sea $O(m \log n)$.

DecreaseKey tiene cota probada de $O(\log n)$ en promedio: cortar y fusionar un nodo es $O(1)$ en el momento, pero puede subir el potencial (más árboles sueltos colgando de la raíz), y esa subida se cobra como parte del costo promedio. En la práctica se cree que cuesta $O(1)$ en promedio y así se comporta experimentalmente, pero nadie ha demostrado formalmente una cota mejor que $O(\log n)$ [1].

Con n inserciones iniciales y q operaciones más, el tiempo total es $O((n + q) \log(n + q))$, de esta forma se está cumpliendo el límite de 2 segundos para $n, q \leq 3 \times 10^5$ como se indica en codeforces.

5 Conclusiones

El pairing heap resuelve el problema sin usar un heap binario: al ser un árbol donde cada nodo puede tener varios hijos, permite sacar el mínimo eficientemente y cortar cualquier nodo en tiempo constante para bajar su clave, todo apoyado en una sola operación básica (*merge*) reutilizada en las tres operaciones pedidas. Su implementación es simple, su velocidad promedio está demostrada matemáticamente, y en la práctica funciona tan bien como estructuras más complicadas.

References

- [1] M. L. Fredman, R. Sedgwick, D. D. Sleator, R. E. Tarjan, "The pairing heap: a new form of self-adjusting heap," *Algorithmica*, 1986.